

QUIZ RICAPITOLAZIONE FRONTEND

Quale delle seguenti affermazioni è corretta riguardo i collegamenti ipertestuali in HTML? Scegli un'alternativa:

- Il tag `<a>` è utilizzato per creare collegamenti ipertestuali
- Il tag `<link>` è utilizzato per creare collegamenti ipertestuali
- Il tag `<href>` è utilizzato per specificare la destinazione di un collegamento ipertestuale
- Il tag `<url>` è utilizzato per creare collegamenti ipertestuali

L'affermazione corretta tra quelle elencate è:

Il tag `<a>` è utilizzato per creare collegamenti ipertestuali

Questa è l'opzione corretta perché:

1. In HTML, il tag `<a>` (abbreviazione di "anchor") è effettivamente utilizzato per creare collegamenti ipertestuali.
2. L'attributo `href` (non un tag) viene utilizzato all'interno del tag `<a>` per specificare la destinazione del collegamento.
3. Il tag `<link>` esiste in HTML, ma non è usato per creare collegamenti ipertestuali visibili nella pagina. È invece usato principalmente nell'intestazione (`<head>`) del documento per collegare fogli di stile esterni o definire relazioni con altre risorse.
4. Non esiste un tag `<href>` in HTML. Come menzionato, `href` è un attributo, non un tag.
5. Non esiste un tag `<url>` in HTML per creare collegamenti ipertestuali.

Un esempio di utilizzo corretto del tag `<a>` per creare un collegamento ipertestuale sarebbe:

```
html
Copy
<a href="https://www.esempio.com">Visita il nostro sito</a>
```

Come si può aggiungere il CSS alle pagina web? Scegli un'alternativa:

- Utilizzo dello stile inline
- Utilizzo dello stile embedded
- Utilizzo del foglio di stile esterno
- In tutti e tre i modi

La risposta corretta a questa domanda è:

In tutti e tre i modi

Questa è l'opzione corretta perché effettivamente ci sono tre modi principali per aggiungere CSS (Cascading Style Sheets) a una pagina web, e tutti e tre sono validi e ampiamente utilizzati. Vediamoli nel dettaglio:

1. Utilizzo dello stile inline: Si applica direttamente a un elemento HTML utilizzando l'attributo `style`.

Esempio:

```
html
Copy
<p style="color: blue; font-size: 16px;">Questo è un paragrafo con stile inline.</p>
```

2. Utilizzo dello stile embedded (interno): Si inserisce all'interno del documento HTML, nella sezione `<head>`, utilizzando il tag `<style>`. Esempio:

```
html
Copy
<head>
  <style>
    p {
      color: blue;
      font-size: 16px;
    }
  </style>
</head>
```

3. Utilizzo del foglio di stile esterno: Si crea un file CSS separato e lo si collega al documento HTML utilizzando il tag `<link>` nella sezione `<head>`. Esempio:

```
html
Copy
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

Ogni metodo ha i suoi vantaggi e svantaggi, e la scelta dipende spesso dalle esigenze specifiche del progetto. In generale, l'uso di fogli di stile esterni è considerato la migliore pratica per progetti di grandi dimensioni, in quanto favorisce la separazione tra struttura (HTML) e presentazione (CSS), migliorando la manutenibilità e la riusabilità del codice.

Questa sintassi in JavaScript: `const somma = (a, b) => a + b;`

Scegli una o più alternative:

- *È una funzione anonima*
- *È scritta seguendo la sintassi della arrow function*
- *Non restituisce nulla perché non c'è l'istruzione `return`*
- *È una dichiarazione di variabile*

Analizziamo questa sintassi JavaScript per determinare quali affermazioni sono corrette:

```
const somma = (a, b) => a + b;
```

Le affermazioni corrette sono:

1. È scritta seguendo la sintassi della arrow function
2. È una dichiarazione di variabile

Spieghiamo perché:

1. È scritta seguendo la sintassi della arrow function: Questa è corretta. La sintassi `(a, b) => a + b` è infatti un esempio di arrow function (funzione freccia), introdotta in ES6.
2. È una dichiarazione di variabile: Anche questa è corretta. L'intera espressione inizia con `const somma =`, che è una dichiarazione di variabile costante.

Per quanto riguarda le altre opzioni:

3. "È una funzione anonima": Tecnicamente, questa non è del tutto corretta. Sebbene l'arrow function in sé sia anonima, viene immediatamente assegnata alla variabile `somma`, dandole quindi un nome.
4. "Non restituisce nulla perché non c'è l'istruzione `return`": Questa affermazione non è corretta. Nelle arrow function, quando il corpo della funzione è una singola espressione (come in questo caso), il `return` è implicito. Quindi questa funzione restituisce effettivamente il risultato di `a + b`.

In conclusione, questa sintassi rappresenta una dichiarazione di variabile costante (`somma`) a cui viene assegnata una arrow function che prende due parametri e restituisce implicitamente la loro somma.

Quale metodo JavaScript può essere utilizzato per cambiare il contenuto di un elemento con `id="pippo"`?

Scegli un'alternativa:

- `document.getElementById("pippo").innerHTML = "Nuovocontenuto";`
- `document.querySelector("pippo").textContent = "Nuovo contenuto";`
- `document.getElementById("pippo").value = "Nuovo contenuto";`
- `document.createElement("pippo").innerHTML = "Nuovo contenuto";`

La risposta corretta è:

```
document.getElementById("pippo").innerHTML = "Nuovocontenuto";
```

Ecco perché questa è l'opzione corretta e le altre non lo sono:

1. `document.getElementById("pippo").innerHTML = "Nuovocontenuto";`
 - Questa è la sintassi corretta per cambiare il contenuto di un elemento HTML con un ID specifico.
 - `getElementById()` seleziona l'elemento con l'ID "pippo".
 - `innerHTML` permette di modificare il contenuto HTML dell'elemento selezionato.
2. `document.querySelector("pippo").textContent = "Nuovo contenuto";`
 - Questa opzione è quasi corretta, ma ha due problemi:
 1. `querySelector()` richiede un selettore CSS. Per selezionare un ID, dovrebbe essere `#pippo`.
 2. `textContent` cambierebbe solo il testo, non l'HTML.
3. `document.getElementById("pippo").value = "Nuovo contenuto";`
 - Questa opzione non è corretta per elementi HTML generici.
 - `value` è utilizzato principalmente per elementi di input come `<input>`, `<textarea>`, o `<select>`.
4. `document.createElement("pippo").innerHTML = "Nuovo contenuto";`
 - Questa opzione crea un nuovo elemento invece di modificarne uno esistente.
 - `createElement()` crea un nuovo elemento, non seleziona uno esistente.

In conclusione, per cambiare il contenuto di un elemento esistente con `id="pippo"`, la prima opzione è la più appropriata e corretta.

Quali affermazioni sono corrette riguardo l'uso delle funzioni in JavaScript? Scegli una o più alternative:

- Le funzioni possono essere definite usando la parola chiave `function`
- Le funzioni in JavaScript non possono restituire un valore
- Le funzioni possono essere passate come argomenti ad altre funzioni
- Le funzioni non possono essere assegnate a variabili

Analizziamo ciascuna affermazione per determinare quali sono corrette riguardo l'uso delle funzioni in JavaScript:

1. Le funzioni possono essere definite usando la parola chiave `function`
 - Questa affermazione è corretta. In JavaScript, una delle modalità più comuni per definire una funzione è utilizzare la parola chiave `function`. Ad esempio:

```
javascript
Copy
function saluta(nome) {
    return "Ciao, " + nome + "!";
}
```

2. Le funzioni in JavaScript non possono restituire un valore
 - Questa affermazione è falsa. Le funzioni in JavaScript possono restituire valori utilizzando la parola chiave `return`. Possono restituire qualsiasi tipo di dato, inclusi oggetti, array, altre funzioni, o valori primitivi.
3. Le funzioni possono essere passate come argomenti ad altre funzioni
 - Questa affermazione è corretta. In JavaScript, le funzioni sono "cittadini di prima classe", il che significa che possono essere trattate come qualsiasi altro valore. Possono essere passate come argomenti ad altre funzioni, il che è alla base di concetti come le callback e le funzioni di ordine superiore.
4. Le funzioni non possono essere assegnate a variabili
 - Questa affermazione è falsa. In JavaScript, le funzioni possono essere assegnate a variabili. Questo è un aspetto fondamentale del linguaggio e permette di creare funzioni anonime e espressioni di funzione. Ad esempio:

```
javascript
Copy
const saluta = function(nome) {
    return "Ciao, " + nome + "!";
};
```

Quindi, le affermazioni corrette sono:

1. Le funzioni possono essere definite usando la parola chiave `function`
2. Le funzioni possono essere passate come argomenti ad altre funzioni

Queste caratteristiche rendono JavaScript un linguaggio molto flessibile e potente quando si tratta di lavorare con le funzioni, permettendo approcci di programmazione funzionale e altri pattern avanzati.

Quali delle seguenti affermazioni riguardo i CSS sono vere? Scegli una o più alternative:

- CSS è un linguaggio di markup
- CSS è utilizzato per definire lo stile di una pagina web
- CSS è un linguaggio di programmazione
- CSS significa Cascading Style Sheets

Le seguenti affermazioni riguardo i CSS sono vere:

- CSS è utilizzato per definire lo stile di una pagina web
- CSS significa Cascading Style Sheets

Spiegazione

- **CSS è un linguaggio di markup:** Falso. CSS (Cascading Style Sheets) è un linguaggio utilizzato per descrivere l'aspetto e la formattazione di un documento scritto in un linguaggio di markup come HTML, ma CSS stesso non è un linguaggio di markup.
- **CSS è utilizzato per definire lo stile di una pagina web:** Vero. CSS viene utilizzato per specificare come gli elementi HTML devono essere visualizzati su schermo, carta o in altri media.
- **CSS è un linguaggio di programmazione:** Falso. CSS non è un linguaggio di programmazione; è un linguaggio di stile che descrive la presentazione dei documenti scritti in HTML o XML.
- **CSS significa Cascading Style Sheets:** Vero. CSS sta per Cascading Style Sheets, che significa "fogli di stile a cascata".

Quale affermazione è corretta riguardo i tag e gli attributi HTML? Scegli un'alternativa:

- Gli attributi HTML vengono sempre scritti all'interno dei tag di chiusura
- Tutti i tag HTML devono avere almeno un attributo

- Gli attributi HTML forniscono informazioni aggiuntive sui tag HTML
- I tag HTML non possono essere nidificati, ovverosia non si può mettere un elemento dentro un'altro elemento.

L'affermazione corretta riguardo i tag e gli attributi HTML è:

- **Gli attributi HTML forniscono informazioni aggiuntive sui tag HTML**

Spiegazione delle altre alternative

- **Gli attributi HTML vengono sempre scritti all'interno dei tag di chiusura:** Falso. Gli attributi HTML vengono sempre scritti all'interno dei tag di apertura, non nei tag di chiusura.
- **Tutti i tag HTML devono avere almeno un attributo:** Falso. Non tutti i tag HTML devono avere attributi. Molti tag possono essere utilizzati senza alcun attributo.
- **I tag HTML non possono essere nidificati, ovverosia non si può mettere un elemento dentro un'altro elemento:** Falso. I tag HTML possono essere nidificati, il che significa che è possibile mettere un elemento HTML dentro un altro elemento HTML. Ad esempio, puoi avere un tag `<div>` che contiene un tag `<p>`.

Quali affermazioni sono corrette riguardo la seguente istruzione?

```
document.getElementById("my-button").addEventListener('click', handleClick)
```

Scegli una o più alternative:

- Permette di intercettare e gestire l'evento di click su un elemento della pagina
- È scritta in sintassi HTML
- `handleClick` è una funzione che verrà eseguita al click sull'elemento con id "my-button"
- `handleClick` è una variabile che contiene un elemento della pagina

Le affermazioni corrette riguardo la seguente istruzione:

```
document.getElementById("my-button").addEventListener('click', handleClick)
```

sono:

- **Permette di intercettare e gestire l'evento di click su un elemento della pagina**
- **`handleClick` è una funzione che verrà eseguita al click sull'elemento con id "my-button"**

Spiegazione delle altre alternative

- **È scritta in sintassi HTML:** Falso. Questa istruzione è scritta in JavaScript, non in HTML. JavaScript viene utilizzato per aggiungere comportamenti dinamici alle pagine web.
- **`handleClick` è una variabile che contiene un elemento della pagina:** Falso. In questo contesto, `handleClick` è una funzione (o dovrebbe essere una funzione definita altrove nel codice) che verrà chiamata quando l'elemento con id "my-button" viene cliccato.

Come si inserisce un commento nel codice HTML? Scegli un'alternativa:

- `/* COMMENTO */`
- `// COMMENTO //`
- `<!-- COMMENTO -->`
- `# COMMENTO #`

La modalità corretta per inserire un commento nel codice HTML è:

- **`<!-- COMMENTO -->`**

Spiegazione

- **`<!-- COMMENTO -->`:** Corretto. In HTML, i commenti sono racchiusi tra `<!--` e `-->`.

Esempio

```
<!-- Questo è un commento in HTML -->
<p>Questo è un paragrafo.</p>
```

Le altre alternative (`/* COMMENTO */`, `// COMMENTO //`, e `# COMMENTO #`) sono usate per commenti in altri linguaggi di programmazione come CSS, JavaScript, Python, ecc., ma non sono valide per HTML.

- *Quale attributo dei tag HTML è utilizzato dai CSS per applicare uno stile specifico ad un gruppo di elementi? Scegli un'alternativa:*
- `id`
- `class`
- `tag`
- `style`

L'attributo dei tag HTML utilizzato dai CSS per applicare uno stile specifico ad un gruppo di elementi è:

- **`class`**

Spiegazione

- **`id`:** L'attributo `id` è utilizzato per identificare un singolo elemento univoco nella pagina e viene generalmente utilizzato nei CSS per applicare uno stile specifico a un elemento unico.

- **class:** Corretto. L'attributo `class` è utilizzato per applicare uno stile specifico a un gruppo di elementi. Gli elementi con la stessa classe avranno lo stesso stile applicato.
- **tag:** Questo non è un attributo. I tag HTML stessi (`<div>`, `<p>`, `<span>`, ecc.) sono utilizzati per strutturare il contenuto della pagina.
- **style:** L'attributo `style` è utilizzato per applicare stili inline direttamente su un singolo elemento, non su un gruppo di elementi.

Esempio di Utilizzo dell'Attributo `class`

```
<div class="my-class">Elemento 1</div>
<div class="my-class">Elemento 2</div>
<div class="my-class">Elemento 3</div>
```

CSS:

css

Copia codice

```
.my-class {
  color: blue;
  font-size: 20px;
}
```

Quale proprietà nei CSS serve ad impostare la dimensione del testo? Scegli un'alternativa:

- `font-size`
- `font-dimension`
- `text-dimension`
- `text-size`

La proprietà nei CSS utilizzata per impostare la dimensione del testo è:

- **`font-size`**

Spiegazione

- **`font-size`:** Corretto. Questa proprietà CSS è utilizzata per impostare la dimensione del testo di un elemento.

Esempio

```
p {
  font-size: 16px;
}
```

- **`font-dimension`:** Non esiste una proprietà CSS con questo nome.
- **`text-dimension`:** Non esiste una proprietà CSS con questo nome.
- **`text-size`:** Non esiste una proprietà CSS con questo nome.

Quale fra queste è una delle caratteristiche di CSS? Scegli un'alternativa:

- *Permette di manipolare i dati nel DOM direttamente*
- *Viene utilizzato per definire lo stile degli elementi HTML*
- *È un linguaggio di scripting*
- *Nessuna delle precedenti*

Una delle caratteristiche di CSS è:

- **Viene utilizzato per definire lo stile degli elementi HTML**

Spiegazione

- **Permette di manipolare i dati nel DOM direttamente:** Falso. La manipolazione diretta del DOM viene eseguita con JavaScript, non con CSS.
- **Viene utilizzato per definire lo stile degli elementi HTML:** Corretto. CSS (Cascading Style Sheets) è utilizzato per definire l'aspetto e il layout degli elementi HTML.
- **È un linguaggio di scripting:** Falso. CSS non è un linguaggio di scripting, ma un linguaggio di descrizione dello stile.
- **Nessuna delle precedenti:** Falso, poiché una delle precedenti affermazioni è corretta.

Quale tra questi tag ha la dimensione maggiore del testo? Scegli un'alternativa:

- `<p></p>`
- `<h3></h3>`
- `<h1></h1>`
- `<h6></h6>`

Tra questi tag, quello con la dimensione maggiore del testo è:

- **`<h1></h1>`**

Spiegazione

- **<p></p>**: Il tag <p> è utilizzato per i paragrafi e generalmente ha una dimensione di testo standard.
- **<h3></h3>**: Il tag <h3> è utilizzato per i titoli di terzo livello e ha una dimensione di testo più piccola rispetto a <h1>.
- **<h1></h1>**: Corretto. Il tag <h1> è utilizzato per i titoli di primo livello e ha la dimensione di testo maggiore rispetto a tutti gli altri tag di intestazione (<h2>, <h3>, <h4>, <h5>, <h6>).
- **<h6></h6>**: Il tag <h6> è utilizzato per i titoli di sesto livello e ha la dimensione di testo più piccola tra tutti i tag di intestazione.

Quali di questi tag non ha a che fare con le tabelle HTML? Scegli una o più alternative:

- **<tr></tr>**
- **<hr/> <td>**
- **</td>**
- ****

I tag che non hanno a che fare con le tabelle HTML sono:

- **<hr/>**
- ****

Spiegazione

- **<tr></tr>**: Questo tag è utilizzato per definire una riga in una tabella HTML.
- **<hr/>**: Corretto. Questo tag è utilizzato per inserire una linea orizzontale nella pagina e non è correlato alle tabelle HTML.
- **<td></td>**: Questo tag è utilizzato per definire una cella in una riga di una tabella HTML.
- ****: Corretto. Questo tag è utilizzato per definire un elemento in una lista (ordinata o non ordinata) e non è correlato alle tabelle HTML.

Quale affermazione è corretta riguardo alla funzione fetch() JavaScript? Scegli una o più alternative:

- *fetch() è utilizzato per inviare richieste HTTP e gestire risposte in JavaScript*
- *fetch() serve per la manipolazione del DOM*
- *fetch() è una funzione asincrona e restituisce una "Promise"*
- *fetch() può comunicare solo in formato JSON*

Le affermazioni corrette riguardo alla funzione fetch() in JavaScript sono:

- **fetch() è utilizzato per inviare richieste HTTP e gestire risposte in JavaScript**
- **fetch() è una funzione asincrona e restituisce una "Promise"**

Spiegazione

- **fetch() è utilizzato per inviare richieste HTTP e gestire risposte in JavaScript**: Corretto. La funzione fetch() viene utilizzata per effettuare richieste HTTP (come GET, POST, PUT, DELETE) e gestire le risposte.
- **fetch() serve per la manipolazione del DOM**: Falso. La funzione fetch() non viene utilizzata per la manipolazione del DOM; questa operazione è eseguita con altre funzioni JavaScript come document.querySelector(), document.getElementById(), ecc.
- **fetch() è una funzione asincrona e restituisce una "Promise"**: Corretto. La funzione fetch() è asincrona e restituisce una Promise, che rappresenta l'eventuale completamento (o fallimento) di un'operazione asincrona e il suo risultato.
- **fetch() può comunicare solo in formato JSON**: Falso. Sebbene fetch() sia spesso utilizzato per lavorare con JSON, può gestire anche altri formati di dati come testo, Blob, FormData, ecc. tramite metodi come response.text(), response.blob(), response.formData(), ecc.

QUIZ RICAPITOLAZIONE BACKEND

Usando il modulo Python "Flask-SQLAlchemy", se l'oggetto `db` è un'istanza di `SQLAlchemy(app)`, quale metodo viene utilizzato per salvare le modifiche sul database al termine delle operazioni? Scegli un'alternativa:

- `db.commit()`
- `db.save()`
- `db.update()`
- `db.flush()`

Usando il modulo Python "Flask-SQLAlchemy", se l'oggetto `db` è un'istanza di `SQLAlchemy(app)`, il metodo utilizzato per salvare le modifiche sul database al termine delle operazioni è:

- `db.commit()`

Spiegazione

- `db.commit()`: Vero. Questo metodo committa le transazioni, rendendo permanenti tutte le modifiche effettuate nella sessione del database.
- `db.save()`: Falso. Anche questo non è un metodo valido per l'oggetto `db` in Flask-SQLAlchemy.
- `db.update()`: Falso. Questo metodo viene utilizzato per aggiornare i record, ma non per salvare le modifiche nel database.
- `db.flush()`: Falso. Questo metodo invia le modifiche in sospeso alla sessione del database, ma non le commit, il che significa che non le salva in modo permanente nel database.

Qual è la funzione utilizzata in SQLAlchemy per creare le tabelle per la prima volta? Scegli un'alternativa:

- `db.create_`
- `all()` `db.create_`
- `table()` `db.create()`
- `db.init_table()`

La funzione utilizzata in SQLAlchemy per creare le tabelle per la prima volta è:

- `db.create_all()`

Spiegazione

- `db.create_all()`: Corretto. Questo metodo crea tutte le tabelle definite nei modelli di SQLAlchemy nel database.
- `db.create_table()`: Falso. Questo non è un metodo valido in SQLAlchemy.
- `db.create()`: Falso. Questo non è un metodo valido in SQLAlchemy.
- `db.init_table()`: Falso. Questo non è un metodo valido in SQLAlchemy.

Quando usi SQLAlchemy con Flask-SQLAlchemy, il metodo `db.create_all()` viene utilizzato per creare tutte le tabelle nel database in base ai modelli definiti.

Esempio

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///mydatabase.db'
db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)

if __name__ == '__main__':
    with app.app_context():
        db.create_all() # Questo crea tutte le tabelle nel database
    app.run(debug=True)
```

In questo esempio, `db.create_all()` crea la tabella `User` nel database `mydatabase.db` la prima volta che viene eseguito.

Che cos'è un dizionario, in Python? Scegli un'alternativa:

- Una "mappatura" ovvero insieme di elementi formati da coppie chiave-valore
- Una sequenza di elementi ordinati formati da coppie chiave-valore
- Una sequenza immutabile di elementi di tipo diverso
- Una sequenza mutabile di elementi di tipo uguale

In Python, un dizionario è:

- Una "mappatura" ovvero insieme di elementi formati da coppie chiave-valore

Spiegazione

- Una "mappatura" ovvero insieme di elementi formati da coppie chiave-valore: Corretto. Un dizionario in Python è una struttura dati che memorizza elementi in coppie chiave-valore. Ogni chiave è univoca e viene utilizzata per accedere al valore corrispondente.

Esempio di Dizionario in Python

```
# Creazione di un dizionario
dizionario = {
    "chiave1": "valore1",
    "chiave2": "valore2",
    "chiave3": "valore3"
}
```

```
# Accesso a un valore tramite la chiave
print(dizionario["chiave1"]) # Output: valore1
```

```
# Aggiunta di una nuova coppia chiave-valore
dizionario["chiave4"] = "valore4"
```

```
# Modifica di un valore esistente
dizionario["chiave1"] = "nuovo_valore1"
```

```
# Rimozione di una coppia chiave-valore
del dizionario["chiave2"]
```

```
# Iterazione attraverso le coppie chiave-valore
for chiave, valore in dizionario.items():
    print(f"{chiave}: {valore}")
```

Perché le altre alternative sono sbagliate

- Una sequenza di elementi ordinati formati da coppie chiave-valore: Falso. Un dizionario non è una sequenza ordinata, ma una mappa. L'ordine delle coppie chiave-valore non è garantito fino a Python 3.7, dove l'ordine di inserimento è preservato per implementazione, ma non è una sequenza ordinata come una lista o una tupla.
- Una sequenza immutabile di elementi di tipo diverso: Falso. Un dizionario è mutabile, non immutabile. Puoi aggiungere, modificare e rimuovere coppie chiave-valore.
- Una sequenza mutabile di elementi di tipo uguale: Falso. Un dizionario può contenere elementi di tipo diverso sia per le chiavi sia per i valori. Non è richiesto che tutti gli elementi siano dello stesso tipo.

Qual è la sintassi corretta per definire una f-string e assegnarla a una variabile? Scegli un'alternativa:

- `mia_stringa == f'Ciao, {name}!'`
- `mia_stringa = f('Ciao, {name}!')`
- `mia_stringa == f"Ciao, {name}!"`
- `mia_stringa = f'Ciao, {name}!'`

La sintassi corretta per definire una f-string e assegnarla a una variabile è:

- `mia_stringa = f'Ciao, {name}!'`

Spiegazione

- `mia_stringa == f'Ciao, {name}!'`: Falso. L'operatore di assegnazione corretto è `=`, non `==`, e `f` non usa parentesi quadre `[ ]` per le espressioni, ma parentesi graffe `{ }`.
- `mia_stringa = f('Ciao, {name}!')`: Falso. La sintassi di una f-string non utilizza parentesi tonde `( )` dopo `f`.
- `mia_stringa == f"Ciao, {name}!"`: Falso. L'operatore di assegnazione corretto è `=`, non `==`.
- `mia_stringa = f'Ciao, {name}!'`: Corretto. Questa è la sintassi corretta per definire una f-string in Python e assegnarla a una variabile.

Esempio

```
name = "Mario"
mia_stringa = f'Ciao, {name}!'
print(mia_stringa) # Output: Ciao, Mario!
```

In questo esempio, `f'Ciao, {name}!'` è una f-string che inserisce il valore della variabile `name` all'interno della stringa.

In generale nel pattern MVC: Scegli una o più alternative:

Il "Modello" rappresenta la struttura e le relazioni della nostra base dati.

La "Vista" rappresenta l'interfaccia grafica dell'applicazione o front-end.

Il "Controller" rappresenta il joystick o il mouse con cui l'utente interagisce con l'applicazione.

Il "Controller" rappresenta la logica dell'applicazione e gestisce le richieste dell'utente.

La "Vista" rappresenta il diagramma di flusso dell'applicazione.

Il "Modello" rappresenta la logica dell'applicazione e gestisce le richieste dell'utente.

Le affermazioni corrette riguardo al pattern MVC (Model-View-Controller) sono:

- Il **"Modello"** rappresenta la struttura e le relazioni della nostra base dati.
- La **"Vista"** rappresenta l'interfaccia grafica dell'applicazione o front-end.
- Il **"Controller"** rappresenta la logica dell'applicazione e gestisce le richieste dell'utente.

Spiegazione

- Il **"Modello"** rappresenta la struttura e le relazioni della nostra base dati.: Corretto. Il Modello (Model) è responsabile della gestione dei dati dell'applicazione, della logica di business e delle regole di business.
- La **"Vista"** rappresenta l'interfaccia grafica dell'applicazione o front-end.: Corretto. La Vista (View) è responsabile della presentazione dei dati e dell'interfaccia utente. È ciò che l'utente vede e con cui interagisce.
- Il **"Controller"** rappresenta il joystick o il mouse con cui l'utente interagisce con l'applicazione.: Falso. Il Controller non rappresenta i dispositivi di input, ma la logica che gestisce le interazioni dell'utente con il sistema.
- Il **"Controller"** rappresenta la logica dell'applicazione e gestisce le richieste dell'utente.: Corretto. Il Controller (Controller) è responsabile della gestione delle richieste dell'utente, dell'interazione con il Modello e della selezione della Vista da mostrare all'utente.
- La **"Vista"** rappresenta il diagramma di flusso dell'applicazione.: Falso. La Vista rappresenta l'interfaccia utente, non un diagramma di flusso.
- Il **"Modello"** rappresenta la logica dell'applicazione e gestisce le richieste dell'utente.: Falso. Il Modello rappresenta la gestione dei dati e le regole di business, non gestisce direttamente le richieste dell'utente.

Quali tra i seguenti NON SONO tipi di dato in Python? Scegli una o più alternative:

- `string`
- `str`
- `int`
- `float`
- `array`
- `boolean`

I tipi di dato che NON SONO tipi di dato in Python tra le seguenti opzioni sono:

- `string`
- `array`
- `boolean`

Spiegazione

- **string**: Non è un tipo di dato in Python. In Python, le stringhe sono rappresentate dal tipo `str`.
- **str**: È un tipo di dato in Python, utilizzato per rappresentare le stringhe.
- **int**: È un tipo di dato in Python, utilizzato per rappresentare i numeri interi.
- **float**: È un tipo di dato in Python, utilizzato per rappresentare i numeri in virgola mobile.
- **array**: Non è un tipo di dato nativo in Python. Python ha le liste (`list`), che sono simili agli array in altri linguaggi, ma `array` non è un tipo di dato nativo di Python. Esiste un modulo chiamato `array` che fornisce supporto per array, ma come tipo di dato nativo `array` non esiste.
- **boolean**: Non è un tipo di dato in Python. In Python, i valori booleani sono rappresentati dal tipo `bool`, che può assumere i valori `True` o `False`.

Qual è il risultato dell'esecuzione del seguente codice Python?

```
def funz(a, b=5, c=10):  
    return a + b * c  
ris = funz(2, 3)
```

`print(ris)`

Scegli un'alternativa:

- 32
- 35
- 52
- 53

Per determinare il risultato dell'esecuzione del codice Python fornito, analizziamo la funzione e la chiamata:

```
def funz(a, b=5, c=10):
```

```
return a + b * c
```

```
ris = funz(2, 3)
print(ris)
```

Spiegazione passo per passo

1. La funzione `funz` prende tre parametri: `a`, `b`, e `c`.
 - `a` è un parametro posizionale obbligatorio.
 - `b` e `c` sono parametri con valori predefiniti (`b=5` e `c=10`).
2. Quando chiamiamo `funz(2, 3)`, stiamo fornendo valori per `a` e `b`.
 - `a = 2`
 - `b = 3`
 - `c` non viene specificato nella chiamata, quindi assume il valore predefinito `10`.
3. La funzione esegue l'operazione `a + b * c`.
 - Sostituendo i valori, otteniamo `2 + 3 * 10`.
 - Seguendo l'ordine delle operazioni (moltiplicazione prima dell'addizione), eseguiamo prima `3 * 10`, che dà `30`.
 - Poi sommiamo `2 + 30`, che dà `32`.

Risultato

Il risultato dell'esecuzione del codice è:

- `32`

In Python, qual è il metodo "costruttore" che viene invocato automaticamente durante la creazione di un'istanza di una classe e che viene utilizzato per inizializzare gli attributi dell'istanza? Scegli un'alternativa:

- `__init__`
- `__new__`
- `__create__`
- `__build__`

In Python, il metodo "costruttore" che viene invocato automaticamente durante la creazione di un'istanza di una classe e che viene utilizzato per inizializzare gli attributi dell'istanza è:

- `__init__`

Spiegazione

- `__init__`: Corretto. Questo metodo speciale viene chiamato automaticamente quando viene creata una nuova istanza della classe. Viene utilizzato per inizializzare gli attributi dell'istanza.
- `__new__`: Questo metodo è un altro metodo speciale in Python che viene chiamato per creare una nuova istanza della classe. Viene eseguito prima di `__init__`, ma non viene utilizzato comunemente per inizializzare gli attributi dell'istanza.
- `__create__`: Non esiste un metodo speciale con questo nome in Python.
- `__build__`: Non esiste un metodo speciale con questo nome in Python.

Esempio di utilizzo di `__init__`

```
class Persona:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta
```

```
# Creazione di un'istanza della classe Persona
p = Persona("Mario", 30)
```

```
# Accesso agli attributi dell'istanza
print(p.nome) # Output: Mario
print(p.eta)  # Output: 30
```

In questo esempio, `__init__` viene utilizzato per inizializzare gli attributi `nome` ed `eta` dell'istanza `p` della classe `Persona`.

Usando il modulo Python "Flask-SQLAlchemy", come si può eseguire una query su un modello `Studiante` per filtrare gli studenti con "altezza > 180", ordinarli per "nome" (a-z) e ottenere il primo record? Scegli un'alternativa:

- `Studiante.query.filter(Studiante.altezza > 180).order_by(Studiante.nome).first()`
- `Studiante.query.filter(Studiante.altezza > 180).order_by(Studiante.nome).get_first()`
- `Studiante.query.filter_by(altezza > 180).order(nome).first()`
- `Studiante.query.filter_by(Studiante.altezza > 180).order_by(nome).first()`

La sintassi corretta per eseguire una query su un modello `Studente` per filtrare gli studenti con "altezza > 180", ordinarli per "nome" (a-z) e ottenere il primo record utilizzando Flask-SQLAlchemy è:

- `Studente.query.filter(Studente.altezza > 180).order_by(Studente.nome).first()`

Spiegazione

- `Studente.query.filter(Studente.altezza > 180).order_by(Studente.nome).first()`: Corretto. Questa è la sintassi corretta per filtrare gli studenti con altezza > 180, ordinarli per nome e ottenere il primo record.
- `Studente.query.filter(Studente.altezza > 180).order_by(Studente.nome).get_first()`: Falso. Non esiste un metodo `get_first()` in Flask-SQLAlchemy.
- `Studente.query.filter_by(altezza > 180).order(nome).first()`: Falso. Il metodo `filter_by` non accetta espressioni di confronto, accetta solo parametri di uguaglianza. Inoltre, `order(nome)` è sintatticamente errato, dovrebbe essere `order_by(Studente.nome)`.
- `Studente.query.filter_by(Studente.altezza > 180).order_by(nome).first()`: Falso. Simile al precedente, `filter_by` non accetta espressioni di confronto. Inoltre, `order_by(nome)` è sintatticamente errato, dovrebbe essere `order_by(Studente.nome)`.

Esempio

Ecco un esempio completo di come eseguire questa query:

```
from flask_sqlalchemy import SQLAlchemy
```

```
db = SQLAlchemy()
```

```
class Studente(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    nome = db.Column(db.String(50))
    altezza = db.Column(db.Float)
```

```
# Query per ottenere il primo studente con altezza > 180 e ordinarlo per nome
studente = Studente.query.filter(Studente.altezza > 180).order_by(Studente.nome).first()
```

```
if studente:
    print(f"Studente trovato: {studente.nome}, Altezza: {studente.altezza}")
else:
    print("Nessuno studente trovato con altezza > 180.")
```

In questo esempio, `Studente.query.filter(Studente.altezza > 180).order_by(Studente.nome).first()` filtra gli studenti con altezza maggiore di 180, li ordina per nome in ordine crescente e restituisce il primo risultato.

Usando il modulo Python "Flask-SQLAlchemy", se l'oggetto `db` è un'istanza di `SQLAlchemy(app)` e `utente` è un'istanza del modello (classe) `Utente`, che cosa fa l'istruzione `db.session.add(utente)`? Scegli un'alternativa:

- Informa la sessione corrente che un nuovo utente `utente` dovrà essere inserito nel database.
- Aggiunge un record `utente` scrivendo le modifiche direttamente sul database.
- Annota nel cookie di sessione che un nuovo utente è stato aggiunto.
- Aggiunge una sessione all'utente indicato.

L'istruzione `db.session.add(utente)` in Flask-SQLAlchemy:

- **Informa la sessione corrente che un nuovo utente `utente` dovrà essere inserito nel database.**

Spiegazione

- **Informa la sessione corrente che un nuovo utente `utente` dovrà essere inserito nel database.**: Corretto. L'istruzione `db.session.add(utente)` aggiunge l'istanza `utente` alla sessione corrente, segnalando che questo oggetto deve essere inserito nel database. Tuttavia, le modifiche non vengono scritte nel database finché non viene eseguito `db.session.commit()`.
- **Aggiunge un record `utente` scrivendo le modifiche direttamente sul database.**: Falso. L'istruzione `db.session.add(utente)` non scrive direttamente le modifiche nel database; è necessario eseguire `db.session.commit()` per applicare le modifiche.
- **Annota nel cookie di sessione che un nuovo utente è stato aggiunto.**: Falso. La sessione a cui si riferisce `db.session` è la sessione del database, non un cookie di sessione.

- **Aggiunge una sessione all'utente indicato.:** Falso. L'istruzione `db.session.add(utente)` non aggiunge una sessione all'utente; piuttosto, aggiunge l'utente alla sessione del database corrente.

Esempio

Ecco un esempio completo di come utilizzare `db.session.add(utente)` insieme a `db.session.commit()`:

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///mydatabase.db'
db = SQLAlchemy(app)

class Utente(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    nome = db.Column(db.String(50))
    email = db.Column(db.String(50))

@app.route('/aggiungi_utente')
def aggiungi_utente():
    nuovo_utente = Utente(nome="Mario", email="mario@example.com")
    db.session.add(nuovo_utente) # Aggiunge l'utente alla sessione del database
    db.session.commit()          # Scrive le modifiche nel database
    return "Utente aggiunto!"

if __name__ == '__main__':
    with app.app_context():
        db.create_all() # Crea le tabelle nel database
    app.run(debug=True)
```

In questo esempio, l'istanza `nuovo_utente` viene aggiunta alla sessione del database con `db.session.add(nuovo_utente)`. Successivamente, l'istruzione `db.session.commit()` salva le modifiche nel database, inserendo effettivamente il nuovo record nella tabella `Utente`.

In Flask, qual è la differenza tra `route` e `endpoint`? Scegli un'alternativa:

- La `route` definisce un URL al quale possono essere inoltrate delle richieste e che di conseguenza esegue la funzione detta di "`endpoint`" per restituire una risposta.
- L'`endpoint` definisce un URL al quale possono essere inoltrate delle richieste e che di conseguenza esegue la funzione detta di "`route`" per restituire una risposta.
- Sono la stessa cosa ed è meglio chiamarli allo stesso modo.
- Sono due cose diverse ma non c'è una differenza precisa.

ChatGPT

La differenza tra "`route`" e "`endpoint`" in Flask è:

- **La `route` definisce un URL al quale possono essere inoltrate delle richieste e che di conseguenza esegue la funzione detta di "`endpoint`" per restituire una risposta.**

Spiegazione

- **La `route` definisce un URL al quale possono essere inoltrate delle richieste e che di conseguenza esegue la funzione detta di "`endpoint`" per restituire una risposta.:** Corretto. In Flask, una "`route`" è un pattern di URL che mappa a una funzione particolare che gestisce le richieste a quell'URL. La funzione stessa è conosciuta come "`endpoint`".
- **L'`endpoint` definisce un URL al quale possono essere inoltrate delle richieste e che di conseguenza esegue la funzione detta di "`route`" per restituire una risposta.:** Falso. È la "`route`" che definisce l'URL, mentre l'"`endpoint`" è la funzione associata che gestisce le richieste.
- **Sono la stessa cosa ed è meglio chiamarli allo stesso modo.:** Falso. Sono concetti correlati ma distinti: la `route` è l'URL pattern e l'`endpoint` è la funzione associata.
- **Sono due cose diverse ma non c'è una differenza precisa.:** Falso. C'è una differenza chiara: la `route` è l'URL pattern e l'`endpoint` è la funzione che risponde a quel pattern.

Esempio

```
from flask import Flask

app = Flask(__name__)

@app.route('/hello')
def hello():
    return 'Hello, World!'
```

In questo esempio:

- **Route:** `'/hello'` è la route, che definisce l'URL al quale possono essere inoltrate delle richieste.
- **Endpoint:** La funzione `hello` è l'endpoint, che gestisce le richieste a `'/hello'` e restituisce una risposta.

Cosa vuol dire che una classe, in Python, può supportare l'ereditarietà multipla? Scegli un'alternativa:

- Che una classe può ereditare da più di una classe genitore
- Che una classe può ereditare sia da una classe genitore sia da una classe figlia
- Che una classe può essere ereditata da più di una classe figlia
- Che una classe può ereditare da più di una classe figlia

In Python, quando si dice che una classe può supportare l'ereditarietà multipla, significa:

- **Che una classe può ereditare da più di una classe genitore**

Spiegazione

- **Che una classe può ereditare da più di una classe genitore:** Corretto. L'ereditarietà multipla permette a una classe di derivare da più di una classe base. Questo consente alla classe derivata di combinare i comportamenti e le proprietà di più classi genitore.
- **Che una classe può ereditare sia da una classe genitore sia da una classe figlia:** Falso. Una classe può ereditare solo da altre classi (genitori), non contemporaneamente da una classe genitore e una classe figlia.
- **Che una classe può essere ereditata da più di una classe figlia:** Falso. Questo descrive una situazione in cui una classe genitore è ereditata da più classi figlie, ma non è ciò che si intende per ereditarietà multipla.
- **Che una classe può ereditare da più di una classe figlia:** Falso. Una classe non eredita dalle sue classi figlie.

Esempio di Ereditarietà Multipla in Python

```
class ClasseA:
    def metodo_a(self):
        return "Metodo da ClasseA"
```

```
class ClasseB:
    def metodo_b(self):
        return "Metodo da ClasseB"
```

```
class ClasseC(ClasseA, ClasseB):
    pass
```

```
# Creazione di un'istanza di ClasseC
c = ClasseC()
print(c.metodo_a()) # Output: Metodo da ClasseA
print(c.metodo_b()) # Output: Metodo da ClasseB
```

In questo esempio, `ClasseC` eredita da entrambe `ClasseA` e `ClasseB`, e quindi può utilizzare i metodi definiti in entrambe le classi genitore.

Quale delle seguenti affermazioni è corretta riguardo le liste in Python? Scegli un'alternativa:

- Le liste sono sequenze mutabili di elementi di tipo diverso
- Le liste sono sequenze immutabili di elementi di tipo diverso
- Le liste sono sequenze mutabili di elementi di tipo uguale
- Le liste sono sequenze immutabili di elementi di tipo uguale

La corretta affermazione riguardo le liste in Python è:

- **Le liste sono sequenze mutabili di elementi di tipo diverso**

Spiegazione

- **Le liste sono sequenze mutabili di elementi di tipo diverso:** Corretto. Le liste in Python possono contenere elementi di tipo diverso (numeri, stringhe, oggetti, ecc.) e possono essere modificate dopo la loro creazione (aggiunta, rimozione o modifica di elementi).
- **Le liste sono sequenze immutabili di elementi di tipo diverso:** Falso. Le liste in Python sono mutabili, il che significa che possono essere modificate dopo la loro creazione.
- **Le liste sono sequenze mutabili di elementi di tipo uguale:** Falso. Le liste in Python possono contenere elementi di tipo diverso, non solo dello stesso tipo.
- **Le liste sono sequenze immutabili di elementi di tipo uguale:** Falso. Le liste in Python sono mutabili e possono contenere elementi di tipo diverso.

Esempio di lista in Python

```
# Creazione di una lista con elementi di tipo diverso
mia_lista = [1, "ciao", 3.14, True]

# Aggiunta di un elemento alla lista
```

```
mia_lista.append("nuovo elemento")
print(mia_lista) # Output: [1, 'ciao', 3.14, True, 'nuovo elemento']
```

```
# Modifica di un elemento nella lista
mia_lista[1] = "salve"
print(mia_lista) # Output: [1, 'salve', 3.14, True, 'nuovo elemento']
```

```
# Rimozione di un elemento dalla lista
del mia_lista[3]
print(mia_lista) # Output: [1, 'salve', 3.14, 'nuovo elemento']
```

In questo esempio, possiamo vedere che la lista `mia_lista` contiene elementi di tipo diverso e può essere modificata dopo la sua creazione.

In Python, quando vedo la funzione `super()` all'interno di un metodo definito in una classe, ad esempio:

```
class Personaggio:
    def saluta():
        print('Ciao!', end=' ')

class Pippo(Personaggio):
    def saluta():
        super().saluta()
        print('Mi chiamo Pippo!')
```

```
istanza_personaggio = Personaggio()
istanza_pippo = Pippo()
istanza_pippo.saluta()
```

Che cosa ne deduco? Cosa verrà stampato quando viene invocato `istanza_pippo.saluta()`? Scegli un'alternativa:

- Ciao! Mi chiamo Pippo!
- Ciao, Pippo!
- Mi chiamo Pippo!
- Ciao!

Quando viene invocato `istanza_pippo.saluta()`, verrà stampato:

- **Ciao! Mi chiamo Pippo!**

Spiegazione

- **Uso di `super()`**: La funzione `super()` è utilizzata per chiamare un metodo della classe genitore all'interno di una classe derivata. In questo caso, `super().saluta()` nella classe `Pippo` chiama il metodo `saluta` della classe `Personaggio`.
- **Ordine delle chiamate**: Quando `istanza_pippo.saluta()` viene chiamato, esegue:
 1. `super().saluta()`, che stampa Ciao! senza andare a capo (`end=' '`).
 2. Poi, continua e stampa Mi chiamo Pippo! sulla stessa riga.

Codice di esempio

```
class Personaggio:
    def saluta(self):
        print('Ciao!', end=' ')

class Pippo(Personaggio):
    def saluta(self):
        super().saluta()
        print('Mi chiamo Pippo!')
```

```
istanza_personaggio = Personaggio()
istanza_pippo = Pippo()
istanza_pippo.saluta()
```

Output

Ciao! Mi chiamo Pippo!

Questo è l'output risultante dalla chiamata `istanza_pippo.saluta()`, che combina il saluto della classe genitore `Personaggio` e il messaggio aggiunto dalla classe derivata `Pippo`.

In Flask, quale metodo viene utilizzato per eseguire l'applicazione web? Scegli un'alternativa:

- `app.run()`
- `app.start()`

- `app.begin()`
- `app.execute()`

In Flask, il metodo utilizzato per eseguire l'applicazione web è:

- **`app.run()`**

Spiegazione

- **`app.run()`**: Corretto. Questo metodo avvia l'applicazione Flask, rendendola pronta per accettare richieste HTTP.
- **`app.start()`**: Falso. Questo non è un metodo valido per eseguire un'applicazione Flask.
- **`app.begin()`**: Falso. Questo non è un metodo valido per eseguire un'applicazione Flask.
- **`app.execute()`**: Falso. Questo non è un metodo valido per eseguire un'applicazione Flask.

Esempio

Ecco un esempio di come utilizzare `app.run()` in un'applicazione Flask:

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')
def home():
    return "Ciao, mondo!"
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

Dettagli dell'Esempio

- **`from flask import Flask`**: Importa la classe Flask.
- **`app = Flask(__name__)`**: Crea un'istanza dell'applicazione Flask.
- **`@app.route('/')`**: Definisce una route per l'URL principale ("/").
- **`def home()`**: Definisce una funzione di vista che restituisce una stringa.
- **`if __name__ == '__main__':`**: Verifica se il file è eseguito come programma principale.
- **`app.run(debug=True)`**: Avvia il server di sviluppo Flask in modalità debug.